Data Analysis Project Report

Team Name: **DataPulse**

Team Members:

1. Ahmed Elkhneny
2. Ammar Dyaa
3. Hamza Salah
4. Mazen Ali
5. Ziad Elazhary

Executive Summary

The UK has a vast train network used daily for work, school, travel, and leisure, including football matches. We analyzed a dataset covering train journeys from January to April 2024, containing details such as transaction ID, ticket type and class, purchase and journey dates, departure and arrival stations, and journey status (on time, delayed, or cancelled), along with delay reasons, durations, and refund requests.
Our goal was to reduce the delay and cancellation rate from 13.2% to 7.6%. We began by asking key questions: What are the main causes of delays? Which stations experience the most delays and cancellations? Using Pandas and Power Query, we cleaned the data, then built visual dashboards in Excel and Tableau to explore these questions—each dashboard highlighting unique insights.

We also examined business performance, identifying the most profitable stations, total profits, and revenue lost due to refunds. Recognizing football's impact on travel demand, we integrated an additional dataset of Premier League and FA Cup matches during the same period. The analysis confirmed that football events significantly increase ticket sales and station activity.
A Pareto analysis revealed that addressing the top three causes of delays and cancellations could reduce these issues by 50.2%. Finally, using Scikit-learn, we built predictive models to forecast ticket sales for the following two months and estimate profit growth if these problems are solved.

Features

<u>Key Features:</u>

1- <u>Comprehensive Data Cleaning and Integration:</u>
Used both *Pandas* and *Power Query* to clean and prepare a multi-parameter dataset of UK train journeys.

2- <u>Dashboard Development:</u>
Created four interactive dashboards using *Excel* and *Tableau*, each designed to highlight unique aspects of the data.

3- <u>Business Analysis:</u>
Explored profitability, refund losses, and financial performance of stations.

4- <u>Delay and Cancellation Analysis:</u>
Focused on identifying the most frequent causes and the stations most affected.

5- <u>Target-Oriented Goal:</u>
Quantified objective—to reduce delay and cancellation rates from 13.2% to 7.6%.

<u>Distinguishable Features</u> :

1- <u>Integration of External Data (Football Matches):</u>
Combined train journey data with Premier League and FA Cup match schedules to analyze how sports events influence passenger flow — a creative and real-world contextual analysis.

2- <u>Multi-Tool Dashboard Ecosystem:</u>
Created dashboards using three different platforms (Excel and Tableau), showcasing versatility and technical breadth.

3- <u>Pareto-Based Root Cause Analysis:</u>
Applied the Pareto principle to identify that solving just three key issues could reduce delays and cancellations by 50.2% turning raw data into actionable strategy.

4- <u>Predictive Modeling for Decision Support:</u>
Used machine learning (Scikit-learn) not just for descriptive insight, but to forecast future performance and estimate potential financial improvement.

Plan

Week	Phase	Key Tasks	Tools	Deliverables
1	Data Collection & Preprocessing	• Collect and merge UK train dataset (Jan–Apr 2024) and football match data. • Clean, preprocess, and validate data to ensure quality and integration.	Excel (Power Query), Python (Pandas)	• Cleaned, integrated dataset ready for analysis. • Data preprocessing notebook/script.
2	Exploratory & Descriptive Analysis	• Explore dataset to identify patterns, data quality issues, and key variables. • Answer main analytical questions regarding delays, cancellations, and refunds.	Python (Pandas, Matplotlib), Excel	• Summary statistics and visual insights (e.g., histograms, box plots). • Identified top delay causes and most affected stations.
3	Business & Event Impact Analysis	• Analyze key business metrics (e.g., profit, refund losses). • Integrate football match data to assess its specific effect on ticket sales and station traffic.	Excel, Tableau	• Report showing detailed business insights. • Visualization of football events impact on train demand/delays.
4	Forecasting & Optimization	• Build predictive models to forecast ticket sales, demand, and profit post-issue resolution. • Perform Pareto analysis to prioritize improvements for maximum delay reduction.	Python (Scikit-learn, Pandas, Matplotlib)	• Forecasting models with performance metrics and visualizations. • Pareto chart showing prioritized areas for potential delay reduction.
5	Dashboard & Final Presentation	• Build a suite of interactive dashboards using Excel, Dash, and Tableau, each focusing on a unique aspect. • Prepare the final project report and presentation summarizing all findings and recommendations.	Excel and Tableau	• Complete dashboard suite (interactive visualizations). • Final Project Report and comprehensive presentation slides.

Problems

1- <u>High Delay and Cancellation Rate:</u>
The overall rate of delayed and cancelled journeys reached **13.2%**, indicating a significant reliability issue across the network. (The Main One)
2- <u>Increased Refund Requests:</u>
Frequent delays and cancellations have led to a high refund rate, directly impacting on revenue and customer satisfaction.
3- <u>Technical and Maintenance Issues:</u>
Equipment failures and operational malfunctions are major contributors to service interruptions.
4- <u>Station and Route Imbalance:</u>
Uneven passenger demand and overloaded routes cause more wear, maintenance delays, and ultimately higher cancellation rates.

Questions

1- What are the most common reasons for train delays and cancellations?
2- Which stations experience the highest frequency of delays and cancellations?
3- What is the average delay duration, categorized by reason and by station?
4- Do delays and cancellations occur more often at specific times of day or on days of the week or month?
5- Are certain routes (Departure–Arrival pairs) more prone to delays and cancellations?
6- How much revenue loss results from delays and cancellations?
7- Which types of ticket or classes (e.g., Standard, Advance, First Class) are most affected by delays?
8- How does the refund request rate vary by station and route?
9- Which stations are the most crowded, based on ticket sales?
10- Which stations and routes generate the highest revenue?
11- What are the most frequently purchased ticket types and classes overall?
12- What is the distribution of payment methods (online vs. station purchase, card vs. cash), and does this affect refund or cancellation rates?
13- How do holidays or work schedule patterns (start and end of workdays) affect train demand?

14- Do special events, such as football matches, cause noticeable changes in ticket sales or delays?

Performance Metrics (KPIs)

KPI	Definition	Formula / Equation
Delay & Cancellation Rate	Measures the overall percentage of train trips that were delayed or cancelled, indicating system unreliability.	(Number of delayed or cancelled trips ÷ Total scheduled trips) × 100
On-Time Performance (OTP)	Indicates the percentage of trips that departed or arrived precisely on schedule, reflecting punctuality.	(Number of on-time trips ÷ Total scheduled trips) × 100
Average Delay Duration	Represents the mean number of minutes each delayed trip was late, showing the severity of disruptions.	(Sum of all delay minutes ÷ Number of delayed trips)
Profit Loss due to Disruptions	Measures the total financial loss directly resulting from delays, cancellations, and associated customer service expenses.	Sum of refunds
Refund Request Rate	The percentage of customers who requested refunds relative to the total number of tickets sold.	(Number of refund requests ÷ Total tickets sold) × 100
Passenger Volume	Tracks the absolute total number of passengers or tickets sold, typically segmented for detailed analysis.	Total tickets sold (per station, route, class/type)
Revenue Contribution	Shows the percentage of total revenue generated by a specific station, route, or ticket type/class.	(Revenue per station/route/class ÷ Total revenue) × 100
High-Demand Event Impact	Compares the delay and cancellation rates during event periods (e.g., football matches) versus normal operating days.	(Delay & cancellation rate during event days) – (Delay & cancellation rate during normal days)

Database Design (Conceptual Schema)

1- Fact_Trip_Performance (Operational Data):

Column Name	Definition	Key Type
Transaction_ID	Unique identifier for each trip record.	PK
Route_ID	References the route of the trip.	FK
Scheduled_Departure_Time	Planned departure time of the trip.	Attribute
Actual_Departure_Time	Actual departure time recorded.	Attribute
Scheduled_Arrival_Time	Planned arrival time of the trip.	Attribute
Actual_Arrival_Time	Actual arrival time recorded.	Attribute
Trip_Status	Indicates status ('On-Time', 'Delayed', Cancelled').	Attribute
Delay_Duration_Minutes	Length of delay in minutes (0 if on time).	Measure
Delay_Reason_ID	Reason for delay or cancellation.	FK
Passenger_Count	Total passengers on the trip (optional metri	Measure

2- Fact_Ticket_Sales (Revenue Data):

Column Name	Definition	Key Type
Ticket_ID	Unique ticket identifier.	PK
Trip_ID	Trip associated with this ticket.	FK
Purchase_Date_ID	Date of purchase.	FK
Ticket_Type_ID	Type/class of the ticket.	FK
Payment_Method_ID	Payment method used.	FK

Column Name	Definition	Key Type
Purchase_Location	Where the ticket was purchased (Online', Station').	Attribute
Ticket_Price	Revenue generated from ticket sale.	Measure

3- Fact_Refunds (Financial Loss Data):

Column Name	Definition	Key Type
Refund_ID	Unique refund identifier.	PK
Ticket_ID	Ticket related to this refund.	FK
Refund_Date_ID	Date when refund was issued.	FK
Refund_Amount	Total amount refunded.	Measure
Refund_Reason_ID	Reason for refund.	FK

4- Dim_Date (Time Context):

Column Name	Definition	Key Type
Date_ID	Unique identifier for each date.	PK
Full_Date	Actual calendar date (YYYY-MM-DD).	Attribute
Day_of_Week	Day name ('Monday', Tuesday', etc.).	Attribute
Month, Quarter, Year	Components for time-based aggregation.	Attribute
Is_Weekend	Flag (TRUE/FALSE).	Attribute

5- Dim_Route (Route Context):

Column Name	Definition	Key Type
Route_ID	Unique identifier for each route.	PK
Departure_Station_ID	Starting station of the route.	FK
Arrival_Station_ID	Ending station of the route.	FK
Average_Travel_Time_Min	Expected duration for the route.	Attribute

6- Dim_Reason (Causal Context):

Column Name	Definition	Key Type
Reason_ID	Unique identifier for each reason.	PK
Reason_Category	Broad category (Technical', 'Weather', 'Staffing').	Attribute
Reason_Description	Detailed explanation of the reason.	Attribute

7- Dim_Ticket_Type (Product Context):

Column Name	**Definition**	Key Type
Ticket_Type_ID	Unique ticket type identifier.	PK
Ticket_Class	Class of ticket ('First', 'Standard').	Attribute
Ticket_Type	Type ('Advance', 'Off-Peak', etc.).	Attribute

8- Dim_Payment_Method (Transaction Context):

Column Name	Definition	Key Type
Payment_Method_ID	Unique payment method identifier.	PK
Payment_Method	Type of payment (‘Credit Card’, ‘Cash’, etc.)	Attribute
Payment_Channel	Source (‘Online’, ‘At Station’).	Attribute

9- Dim_Football_Match (Event Context):

Column Name	Definition	Key Type
Match_ID	Unique identifier for each match.	PK
Match_Date_ID	Date on which the match took place.	FK
Home_Team_City	City of the home team.	Attribute
Away_Team_City	City of the away team.	Attribute
Venue_City	City where the match was played (this is th Dim_Station city/region).	Attribute

Data Cleaning

1- Checking for Missing Values:

```
df.isnull().sum()
```

	0
Transaction ID	0
Date of Purchase	0
Time of Purchase	0
Purchase Type	0
Payment Method	0
Railcard	20918
Ticket Class	0
Ticket Type	0
Price	0
Departure Station	0
Arrival Destination	0
Date of Journey	0
Departure Time	0
Arrival Time	0
Actual Arrival Time	1880
Journey Status	0
Reason for Delay	27481
Refund Request	0

dtype: int64

2- Removing Duplicates:

```
df.drop_duplicates()
```

	Transaction ID	Date of Purchase	Time of Purchase	Purchase Type	Payment Method	Railcard	Ticket Class	Ticket Type	Price	Departure Station	Arrival Destination	Date of Journey	Departure Time	Arrival Time	Actual Arrival Time	Journey Status	Reason for Delay	Refund Request
0	da8a6ba8-b3dc-4677-b176	2023-12-08	12:41:11	Online	Contactless	Adult	Standard	Advance	43	London Paddington	Liverpool Lime Street	2024-01-01	11:00:00	13:30:00	13:30:00	On Time	NaN	No
1	b0cdd1b0-f214-4197-be53	2023-12-16	11:23:01	Station	Credit Card	Adult	Standard	Advance	23	London Kings Cross	York	2024-01-01	09:45:00	11:35:00	11:40:00	Delayed	Signal Failure	No
2	f3ba7a96-f713-40d9-9629	2023-12-19	19:51:27	Online	Credit Card	NaN	Standard	Advance	3	Liverpool Lime Street	Manchester Piccadilly	2024-01-02	18:15:00	18:45:00	18:45:00	On Time	NaN	No
3	b2471f11-4fe7-4c87-8ab4	2023-12-20	23:00:36	Station	Credit Card	NaN	Standard	Advance	13	London Paddington	Reading	2024-01-01	21:30:00	22:30:00	22:30:00	On Time	NaN	No
4	2be00b45-0762-485e-a7a3	2023-12-27	18:22:56	Online	Contactless	NaN	Standard	Advance	76	Liverpool Lime Street	London Euston	2024-01-01	16:45:00	19:00:00	19:00:00	On Time	NaN	No
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...

3- Filtering Valid Purchase Types:

```
df[(df["Purchase Type"] == "Online") | (df["Purchase Type"] == "Station")]
```

	Transaction ID	Date of Purchase	Time of Purchase	Purchase Type	Payment Method	Railcard	Ticket Class	Ticket Type	Price	Departure Station	Arrival Destination	Date of Journey	Departure Time	Arrival Time	Actual Arrival Time	Journey Status	Reason for Delay	Refund Request
0	da8a6ba8-b3dc-4677-b176	2023-12-08	12:41:11	Online	Contactless	Adult	Standard	Advance	43	London Paddington	Liverpool Lime Street	2024-01-01	11:00:00	13:30:00	13:30:00	On Time	NaN	No
1	b0cdd1b0-f214-4197-be53	2023-12-16	11:23:01	Station	Credit Card	Adult	Standard	Advance	23	London Kings Cross	York	2024-01-01	09:45:00	11:35:00	11:40:00	Delayed	Signal Failure	No
2	f3ba7a96-f713-40d9-9629	2023-12-19	19:51:27	Online	Credit Card	NaN	Standard	Advance	3	Liverpool Lime Street	Manchester Piccadilly	2024-01-02	18:15:00	18:45:00	18:45:00	On Time	NaN	No
3	b2471f11-4fe7-4c87-8ab4	2023-12-20	23:00:36	Station	Credit Card	NaN	Standard	Advance	13	London Paddington	Reading	2024-01-01	21:30:00	22:30:00	22:30:00	On Time	NaN	No
4	2be00b45-0762-485e-a7a3	2023-12-27	18:22:56	Online	Contactless	NaN	Standard	Advance	76	Liverpool Lime Street	London Euston	2024-01-01	16:45:00	19:00:00	19:00:00	On Time	NaN	No
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...

4- Filtering Valid Payment Methods:

```
df[df['Payment Method'].isin(["Contactless", "Credit Card", "Debit Card"])]
```

	Transaction ID	Date of Purchase	Time of Purchase	Purchase Type	Payment Method	Railcard	Ticket Class	Ticket Type	Price	Departure Station	Arrival Destination	Date of Journey	Departure Time	Arrival Time	Actual Arrival Time	Journey Status	Reason for Delay	Refund Request
0	da8a6ba8-b3dc-4677-b176	2023-12-08	12:41:11	Online	Contactless	Adult	Standard	Advance	43	London Paddington	Liverpool Lime Street	2024-01-01	11:00:00	13:30:00	13:30:00	On Time	NaN	No
1	b0cdd1b0-f214-4197-be53	2023-12-16	11:23:01	Station	Credit Card	Adult	Standard	Advance	23	London Kings Cross	York	2024-01-01	09:45:00	11:35:00	11:40:00	Delayed	Signal Failure	No
2	f3ba7a96-f713-40d9-9629	2023-12-19	19:51:27	Online	Credit Card	NaN	Standard	Advance	3	Liverpool Lime Street	Manchester Piccadilly	2024-01-02	18:15:00	18:45:00	18:45:00	On Time	NaN	No
3	b2471f11-4fe7-4c87-8ab4	2023-12-20	23:00:36	Station	Credit Card	NaN	Standard	Advance	13	London Paddington	Reading	2024-01-01	21:30:00	22:30:00	22:30:00	On Time	NaN	No
4	2be00b45-0762-485e-a7a3	2023-12-27	18:22:56	Online	Contactless	NaN	Standard	Advance	76	Liverpool Lime Street	London Euston	2024-01-01	16:45:00	19:00:00	19:00:00	On Time	NaN	No
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...

5- Handling Missing Railcard Data:

```
df['Railcard']=df['Railcard'].fillna('none')
df
```

	Transaction ID	Date of Purchase	Time of Purchase	Purchase Type	Payment Method	Railcard	Ticket Class	Ticket Type	Price	Departure Station	Arrival Destination	Date of Journey	Departure Time	Arrival Time	Actual Arrival Time	Journey Status	Reason for Delay	Refund Request
0	da8a6ba8-b3dc-4677-b176	2023-12-08	12:41:11	Online	Contactless	Adult	Standard	Advance	43	London Paddington	Liverpool Lime Street	2024-01-01	11:00:00	13:30:00	13:30:00	On Time	NaN	No
1	b0cdd1b0-f214-4197-be53	2023-12-16	11:23:01	Station	Credit Card	Adult	Standard	Advance	23	London Kings Cross	York	2024-01-01	09:45:00	11:35:00	11:40:00	Delayed	Signal Failure	No
2	f3ba7a96-f713-40d9-9629	2023-12-19	19:51:27	Online	Credit Card	none	Standard	Advance	3	Liverpool Lime Street	Manchester Piccadilly	2024-01-02	18:15:00	18:45:00	18:45:00	On Time	NaN	No
3	b2471f11-4fe7-4c87-8ab4	2023-12-20	23:00:36	Station	Credit Card	none	Standard	Advance	13	London Paddington	Reading	2024-01-01	21:30:00	22:30:00	22:30:00	On Time	NaN	No
4	2be00b45-0762-485e-a7a3	2023-12-27	18:22:56	Online	Contactless	none	Standard	Advance	76	Liverpool Lime Street	London Euston	2024-01-01	16:45:00	19:00:00	19:00:00	On Time	NaN	No
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...

6- Handling Missing Delay Reasons:

```
df['Reason for Delay']=df['Reason for Delay'].fillna('No delay')
df
```

	Transaction ID	Date of Purchase	Time of Purchase	Purchase Type	Payment Method	Railcard	Ticket Class	Ticket Type	Price	Departure Station	Arrival Destination	Date of Journey	Departure Time	Arrival Time	Actual Arrival Time	Journey Status	Reason for Delay	Refund Request
0	da8a6ba8-b3dc-4677-b176	2023-12-08	12:41:11	Online	Contactless	Adult	Standard	Advance	43	London Paddington	Liverpool Lime Street	2024-01-01	11:00:00	13:30:00	13:30:00	On Time	No delay	No
1	b0cdd1b0-f214-4197-be53	2023-12-16	11:23:01	Station	Credit Card	Adult	Standard	Advance	23	London Kings Cross	York	2024-01-01	09:45:00	11:35:00	11:40:00	Delayed	Signal Failure	No
2	f3ba7a96-f713-40d9-9629	2023-12-19	19:51:27	Online	Credit Card	none	Standard	Advance	3	Liverpool Lime Street	Manchester Piccadilly	2024-01-02	18:15:00	18:45:00	18:45:00	On Time	No delay	No
3	b2471f11-4fe7-4c87-8ab4	2023-12-20	23:00:36	Station	Credit Card	none	Standard	Advance	13	London Paddington	Reading	2024-01-01	21:30:00	22:30:00	22:30:00	On Time	No delay	No
4	2be00b45-0762-485e-a7a3	2023-12-27	18:22:56	Online	Contactless	none	Standard	Advance	76	Liverpool Lime Street	London Euston	2024-01-01	16:45:00	19:00:00	19:00:00	On Time	No delay	No
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...

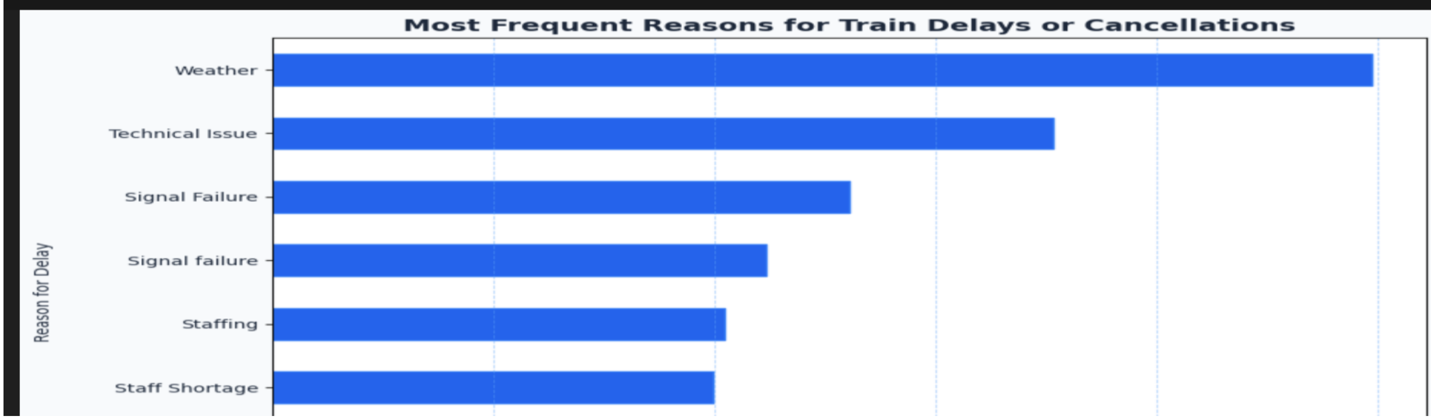
7- Exporting the Cleaned Dataset:

```
df.to_csv("cleaned_data_UK_with_discount.csv", index=False)
from google.colab import files
files.download("cleaned_data_UK_with_discount.csv")
```

Answering Questions & Visualization

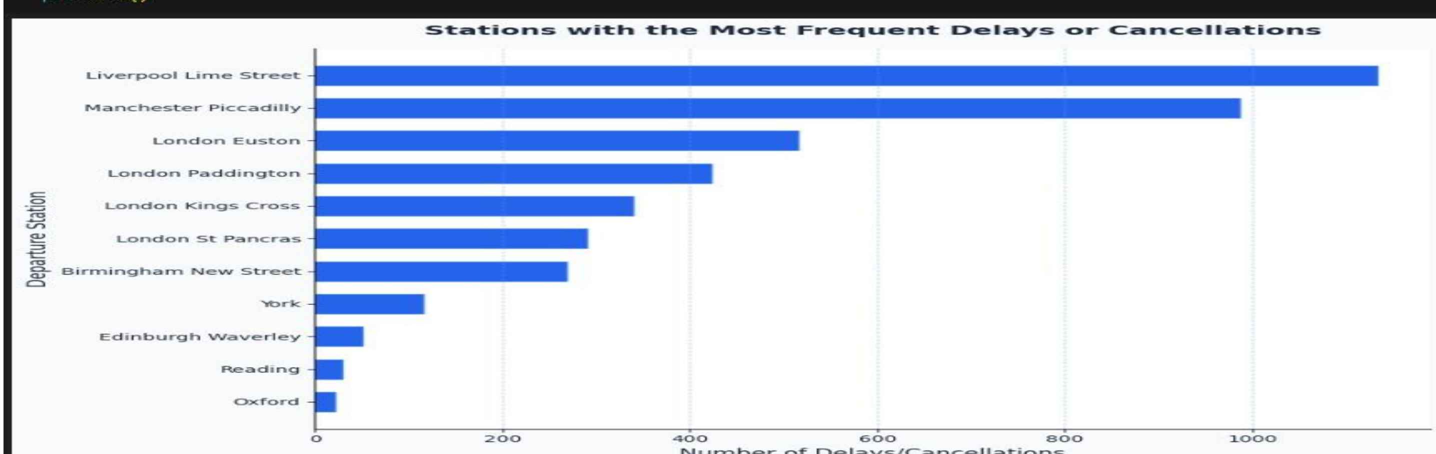
1- What are the most common reasons for train delays and cancellations?

```python
delays = df[df['Reason for Delay'] != 'No delay']
reason_counts = delays['Reason for Delay'].value_counts()
plt.figure(figsize=(10, 7), facecolor=background)
ax = reason_counts.plot(kind='barh',color=primary,edgecolor=secondary)
plt.title("Most Frequent Reasons for Train Delays or Cancellations",color=text_color, fontsize=14, weight='bold')
plt.xlabel("Number of Occurrences", color=text_color)
plt.ylabel("Reason for Delay", color=text_color)
ax.set_facecolor(card_bg)
plt.gca().invert_yaxis()
plt.grid(axis='x', linestyle='--', alpha=0.4, color=accent)
plt.xticks(color=text_color)
plt.yticks(color=text_color)
plt.tight_layout()
plt.show()
```



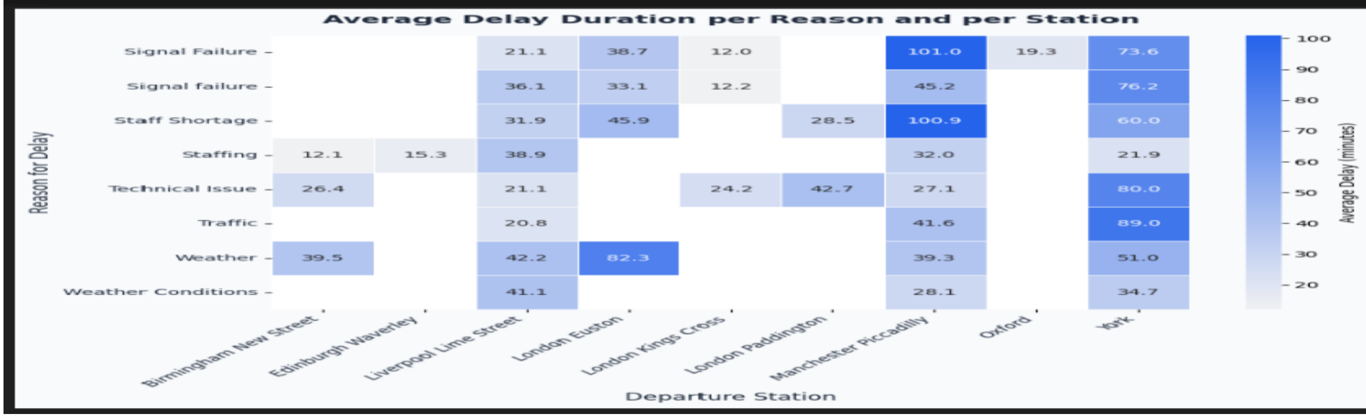
2- Which stations experience the highest frequency of delays and cancellations?

```python
problematic = df[(df["Journey Status"] != "On Time") | (df["Reason for Delay"] != "No delay")]
station_counts = problematic["Departure Station"].value_counts()
fig, ax = plt.subplots(figsize=(10, 7), facecolor=background)
ax.set_facecolor(card_bg)
bars = ax.barh(station_counts.index, station_counts.values,color=primary, edgecolor=secondary, height=0.6)
ax.set_title("Stations with the Most Frequent Delays or Cancellations",fontsize=14, fontweight="bold", color=text_color, pad=15)
ax.set_xlabel("Number of Delays/Cancellations", fontsize=12, color=text_color)
ax.set_ylabel("Departure Station", fontsize=12, color=text_color)
ax.invert_yaxis()
ax.grid(axis="x", linestyle="--", alpha=0.4, color=accent)
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
ax.tick_params(axis="x", colors=text_color)
ax.tick_params(axis="y", colors=text_color)
plt.tight_layout()
plt.show()
```



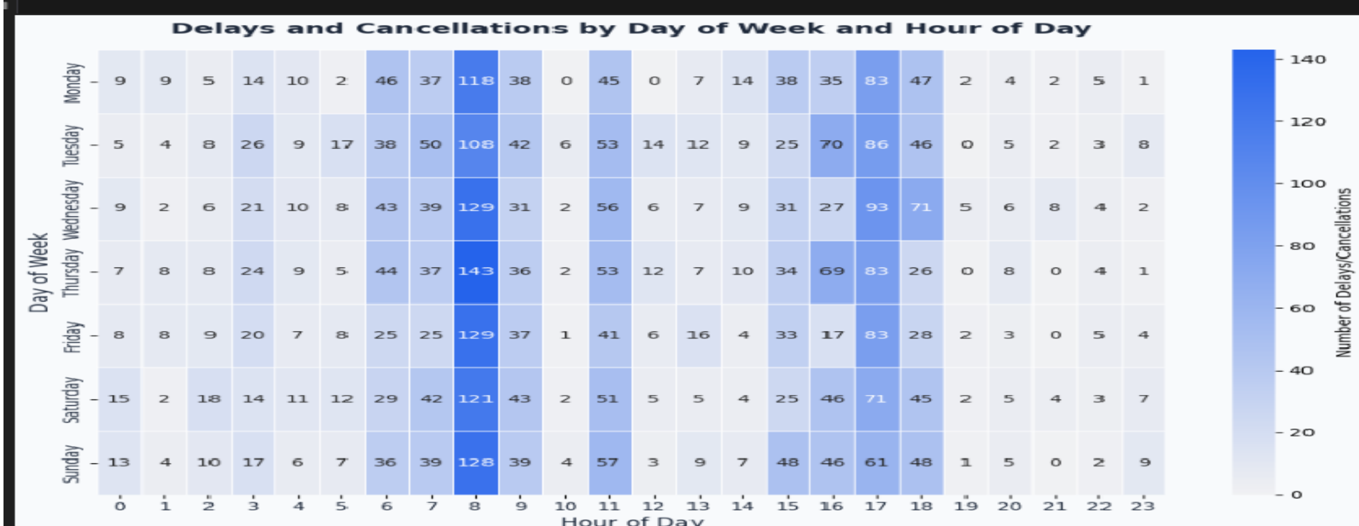
3- What is the average delay duration, categorized by reason and by station?

```
df["Arrival Time"] = pd.to_datetime(df["Arrival Time"], format="%H:%M:%S")
df["Actual Arrival Time"] = pd.to_datetime(df["Actual Arrival Time"], format="%H:%M:%S")
df["Delay_Minutes"] = (df["Actual Arrival Time"] - df["Arrival Time"]).dt.total_seconds() / 60
df["Delay_Minutes"] = df["Delay_Minutes"].clip(lower=0)
delayed = df[df["Delay_Minutes"] > 0]
avg_delay = delayed.groupby(["Reason for Delay", "Departure Station"])["Delay_Minutes"].mean().reset_index()
pivot_table = avg_delay.pivot(index="Reason for Delay", columns="Departure Station", values="Delay_Minutes")

plt.figure(figsize=(10, 7), facecolor=background)
sns.heatmap(pivot_table,annot=True, fmt=".1f", cmap=sns.light_palette(primary, as_cmap=True),
    linewidths=0.5, cbar_kws={"label": "Average Delay (minutes)"}
)
plt.title("Average Delay Duration per Reason and per Station", fontsize=14, fontweight="bold", color=text_color, pad=15)
plt.xlabel("Departure Station", fontsize=12, color=text_color)
plt.ylabel("Reason for Delay", fontsize=12, color=text_color)
plt.xticks(rotation=45, ha="right", color=text_color)
plt.yticks(rotation=0, color=text_color)
plt.tight_layout()
plt.show()
```



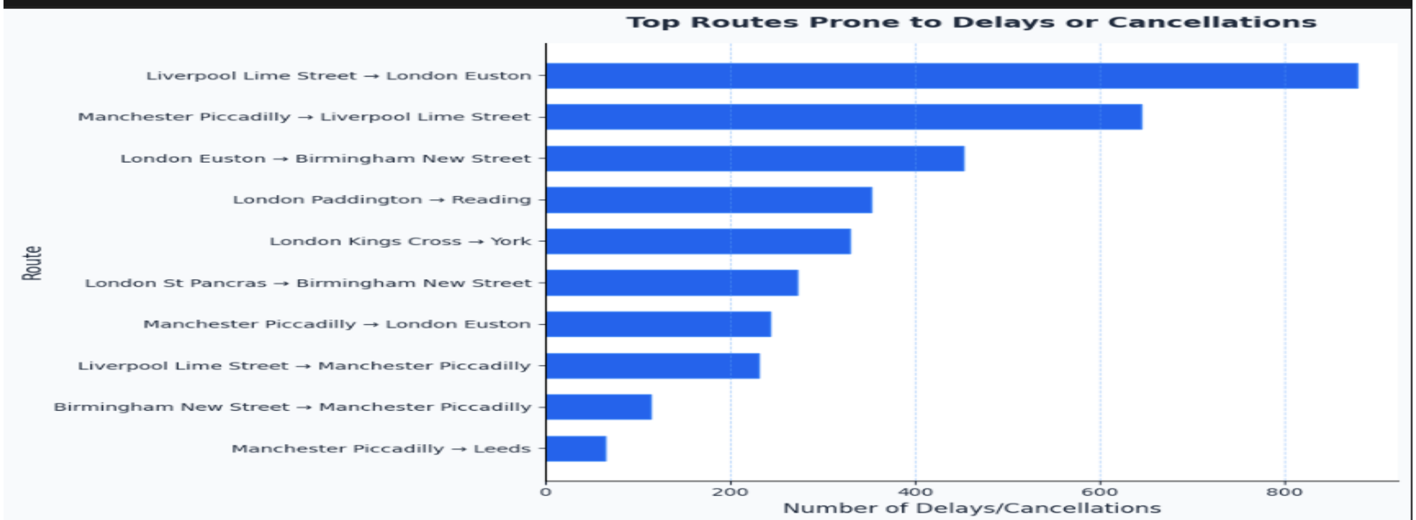
4- Do delays and cancellations occur more often at specific times of day or on days of the week or month?

```
df["Date of Journey"] = pd.to_datetime(df["Date of Journey"])
df["Departure Time"] = pd.to_datetime(df["Departure Time"], format="%H:%M:%S", errors="coerce")
df["DayOfWeek"] = df["Date of Journey"].dt.day_name()
df["Month"] = df["Date of Journey"].dt.strftime("%B")
df["Hour"] = df["Departure Time"].dt.hour
problems = df[(df["Journey Status"] != "On Time") | (df["Reason for Delay"] != "No delay")]
delay_pattern = problems.groupby(["DayOfWeek", "Hour"]).size().reset_index(name="Count")
days_order = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"]
delay_pattern["DayOfWeek"] = pd.Categorical(delay_pattern["DayOfWeek"], categories=days_order, ordered=True)
pivot = delay_pattern.pivot(index="DayOfWeek", columns="Hour", values="Count").fillna(0)
plt.figure(figsize=(10, 7), facecolor=background)
sns.heatmap(
    pivot,
    cmap=sns.light_palette(primary, as_cmap=True),
    annot=True, fmt=".0f",
    linewidths=0.5,
    cbar_kws={"label": "Number of Delays/Cancellations"}
)
plt.title("Delays and Cancellations by Day of Week and Hour of Day",
          fontsize=14, fontweight="bold", color=text_color, pad=15)
plt.xlabel("Hour of Day", fontsize=12, color=text_color)
plt.ylabel("Day of Week", fontsize=12, color=text_color)
plt.xticks(color=text_color)
plt.yticks(color=text_color)
plt.tight_layout()
plt.show()
```



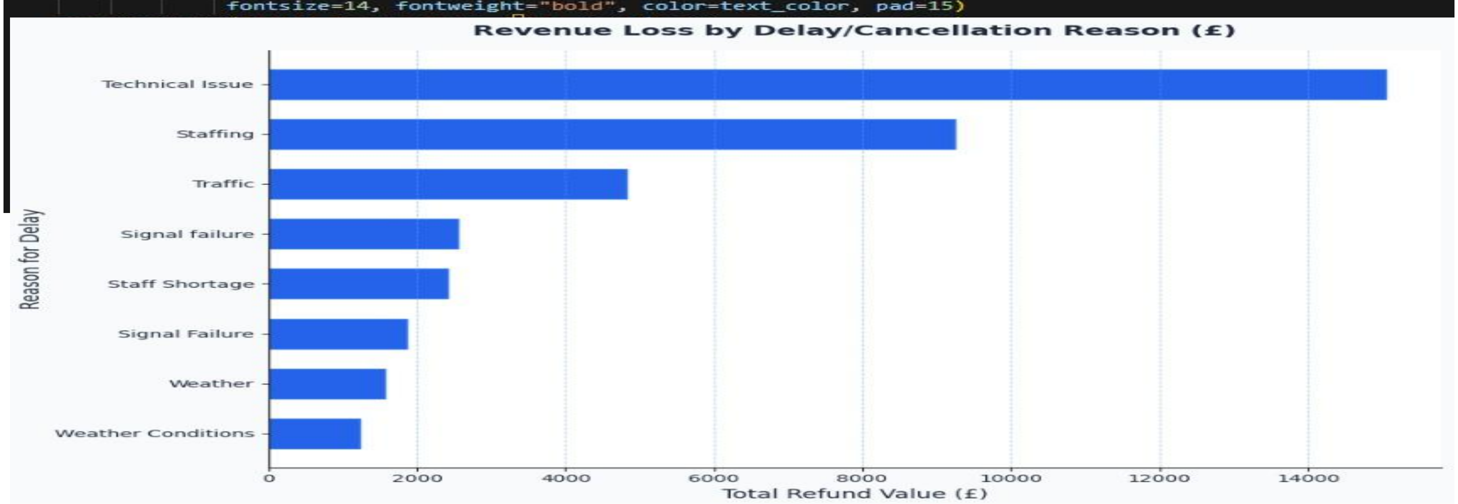
5- Are certain routes (Departure–Arrival pairs) more prone to delays and cancellations?

```
df["Route"] = df["Departure Station"] + " → " + df["Arrival Destination"]
problems = df[(df["Journey Status"] != "On Time") | (df["Reason for Delay"] != "No delay")]
route_counts = problems["Route"].value_counts().head(10)
fig, ax = plt.subplots(figsize=(10,7), facecolor=background)
ax.set_facecolor(card_bg)
bars = ax.barh(route_counts.index, route_counts.values,
               color=primary, edgecolor=secondary, height=0.6)
ax.set_title("Top Routes Prone to Delays or Cancellations",
             fontsize=14, fontweight="bold", color=text_color, pad=15)
ax.set_xlabel("Number of Delays/Cancellations", fontsize=12, color=text_color)
ax.set_ylabel("Route", fontsize=12, color=text_color)
ax.invert_yaxis()
ax.grid(axis="x", linestyle="--", alpha=0.4, color=accent)
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
ax.tick_params(axis="x", colors=text_color)
ax.tick_params(axis="y", colors=text_color)
plt.tight_layout()
plt.show()
```



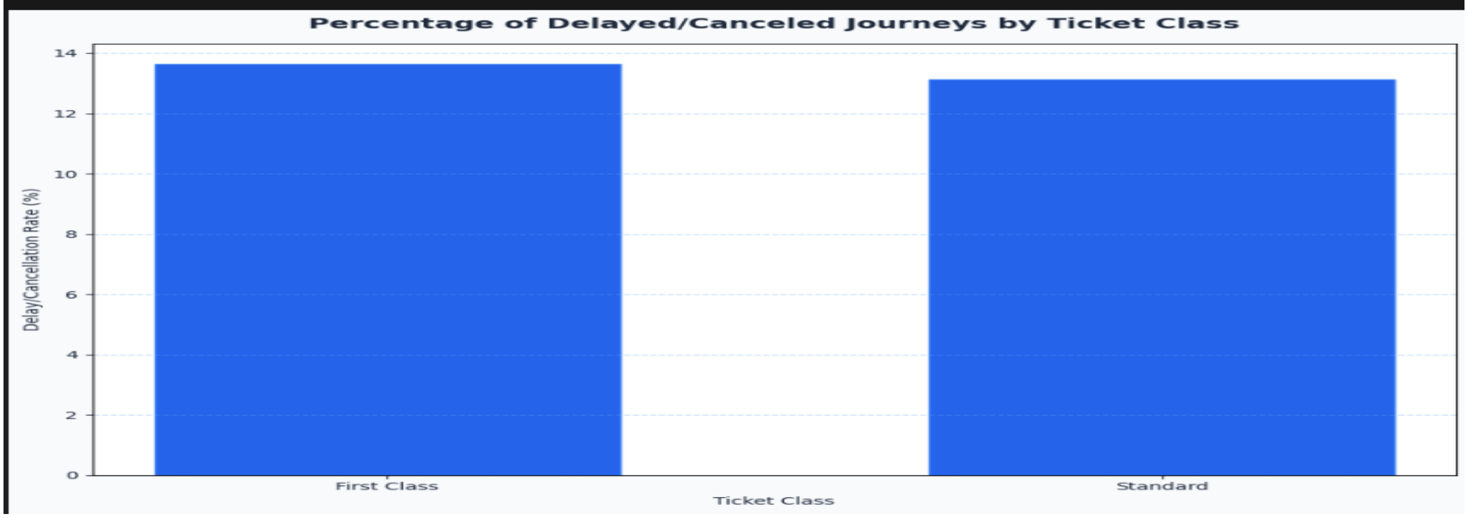
6- How much revenue loss results from delays and cancellations?

```
total_revenue = df["Price"].sum()
refunds = df[df["Refund Request"].str.lower() == "yes"]
revenue_loss = refunds["Price"].sum()
loss_by_reason = refunds.groupby("Reason for Delay")["Price"].sum().sort_values(ascending=False)
fig, ax = plt.subplots(figsize=(10, 7), facecolor=background)
ax.set_facecolor(card_bg)
bars = ax.barh(loss_by_reason.index, loss_by_reason.values,
               color=primary, edgecolor=secondary, height=0.6)
ax.set_title("Revenue Loss by Delay/Cancellation Reason (£)",
             fontsize=14, fontweight="bold", color=text_color, pad=15)
```

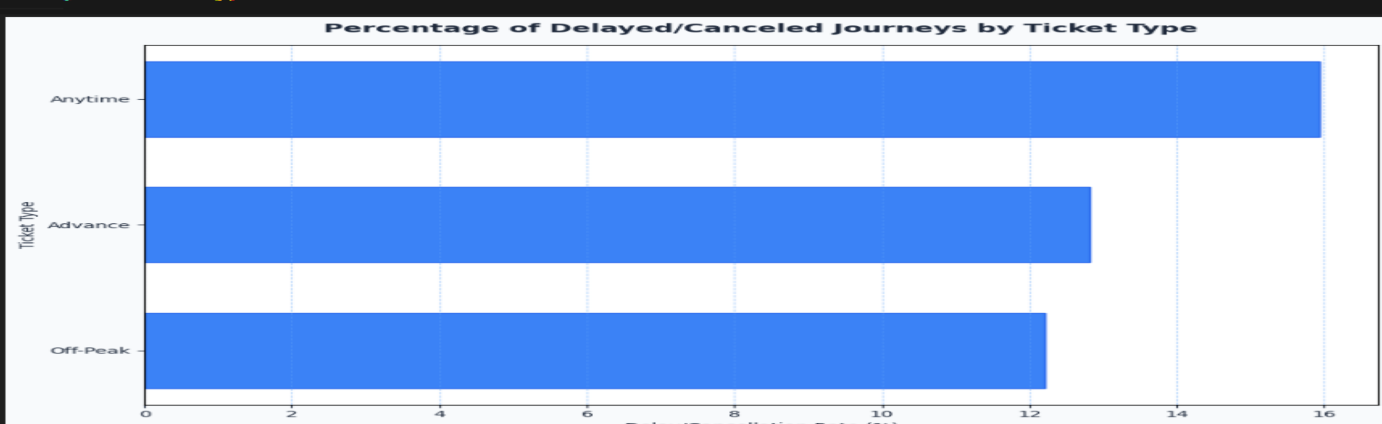


7- Which types of ticket or classes (e.g., Standard, Advance, First Class) are most affected by delays?

```
df["Journey Status"] = df["Journey Status"].str.lower()
df["Ticket Class"] = df["Ticket Class"].fillna("Unknown")
df["Ticket Type"] = df["Ticket Type"].fillna("Unknown")
delay_mask = df["Journey Status"].isin(["delayed", "cancelled"])
delays_by_class = df[delay_mask]["Ticket Class"].value_counts()
total_by_class = df["Ticket Class"].value_counts()

delay_percent_class = (delays_by_class / total_by_class * 100).fillna(0).sort_values(ascending=False)
delays_by_type = df[delay_mask]["Ticket Type"].value_counts()
total_by_type = df["Ticket Type"].value_counts()
delay_percent_type = (delays_by_type / total_by_type * 100).fillna(0).sort_values(ascending=False)
fig, ax = plt.subplots(figsize=(10, 7), facecolor=background)
ax.set_facecolor(card_bg)
bars = ax.bar(delay_percent_class.index, delay_percent_class.values,
              color=primary, edgecolor=secondary, width=0.6)
ax.set_title("Percentage of Delayed/Canceled Journeys by Ticket Class",
             color=text_color, fontsize=14, fontweight="bold", pad=15)
ax.set_xlabel("Ticket Class", color=text_color)
ax.set_ylabel("Delay/Cancellation Rate (%)", color=text_color)
ax.grid(axis="y", linestyle="--", alpha=0.4, color=accent)
ax.tick_params(colors=text_color)
plt.tight_layout()
plt.show()
```

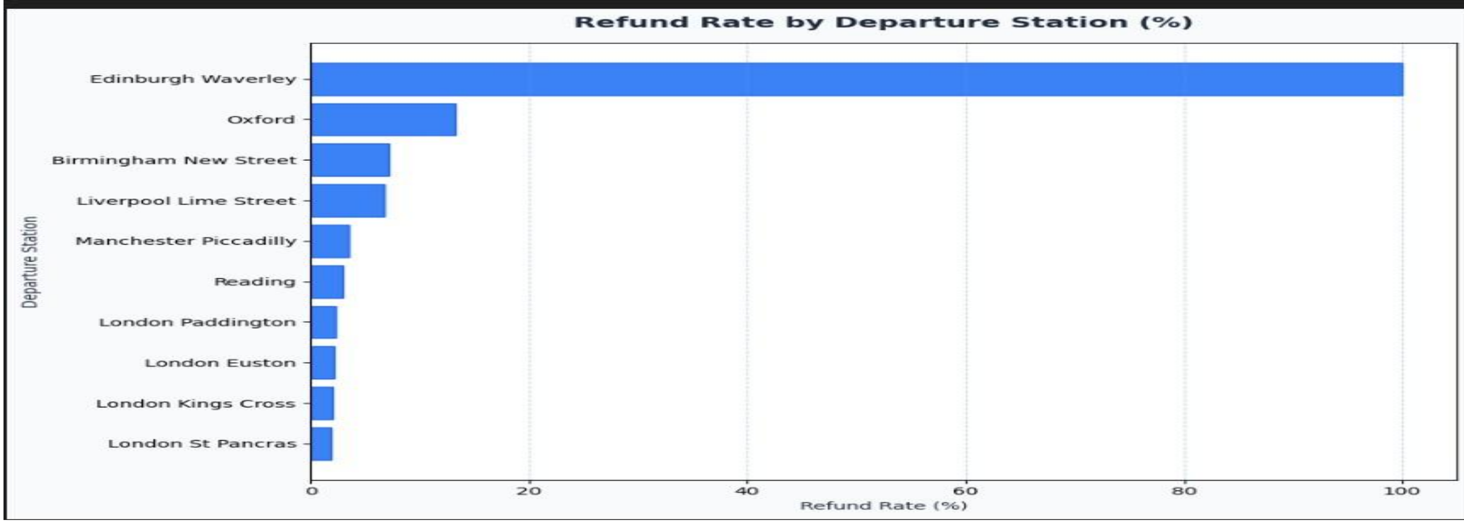



```
fig, ax = plt.subplots(figsize=(10, 7), facecolor=background)
ax.set_facecolor(card_bg)
bars = ax.barh(delay_percent_type.index, delay_percent_type.values,
               color=secondary, edgecolor=primary, height=0.6)
ax.set_title("Percentage of Delayed/Canceled Journeys by Ticket Type",
             color=text_color, fontsize=14, fontweight="bold", pad=15)
ax.set_xlabel("Delay/Cancellation Rate (%)", color=text_color)
ax.set_ylabel("Ticket Type", color=text_color)
ax.grid(axis="x", linestyle="--", alpha=0.4, color=accent)
ax.invert_yaxis()
ax.tick_params(colors=text_color)
plt.tight_layout()
plt.show()
```

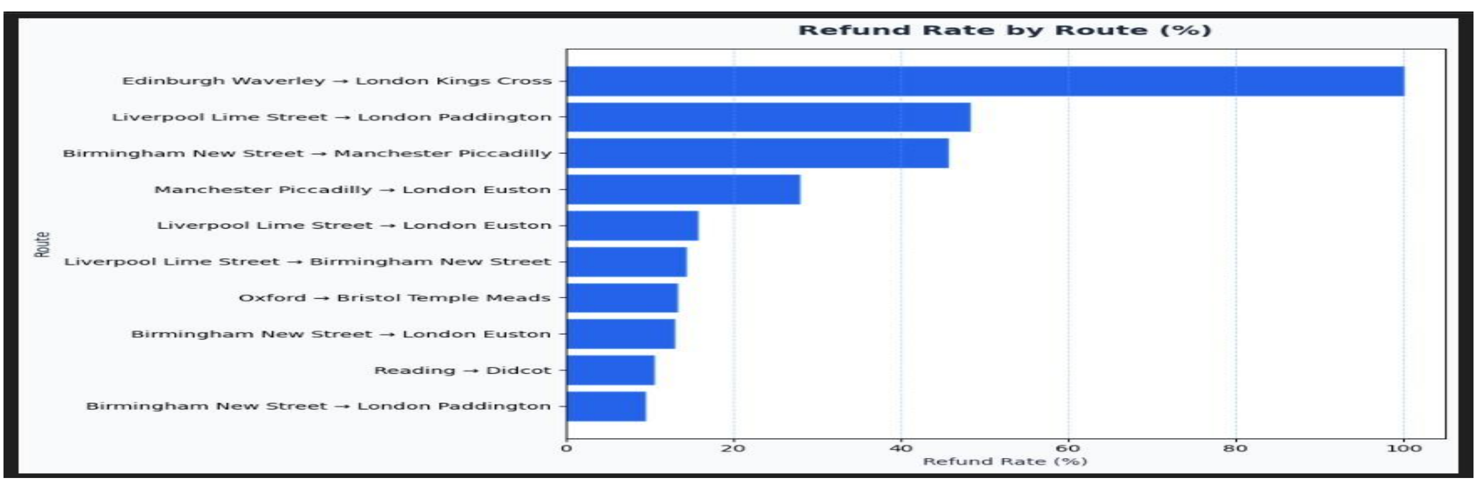


8- How does the refund request rate vary by station and route?

```python
rate_by_station = refund_rate("Departure Station").head(10)
plt.figure(figsize=(10,7), facecolor=background)
bars = plt.barh(rate_by_station.index, rate_by_station["Refund_Rate"],
                color=secondary, edgecolor=primary)
plt.title("Refund Rate by Departure Station (%)", fontsize=14, fontweight="bold", color=text_color, pad=15)
plt.xlabel("Refund Rate (%)", color=text_color)
plt.ylabel("Departure Station", color=text_color)
plt.grid(axis="x", linestyle="--", alpha=0.4, color=accent)
plt.gca().invert_yaxis()
plt.tight_layout()
plt.show()
```

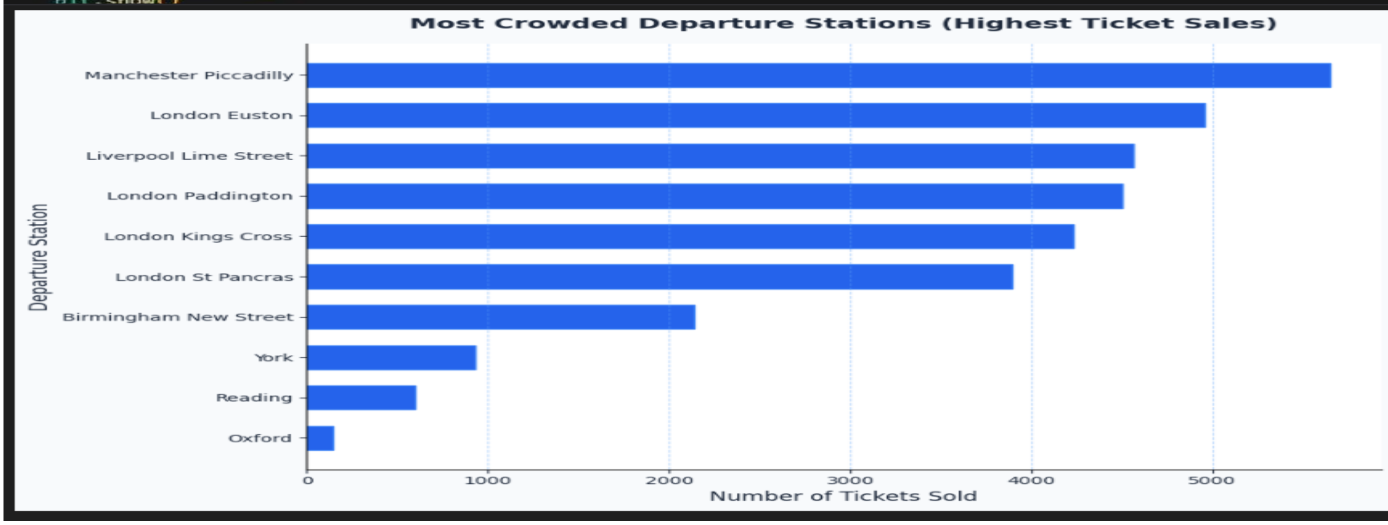



```python
rate_by_route = refund_rate("Route").head(10)
plt.figure(figsize=(10,7), facecolor=background)
bars = plt.barh(rate_by_route.index, rate_by_route["Refund_Rate"],
                color=primary, edgecolor=secondary)
plt.title("Refund Rate by Route (%)", fontsize=14, fontweight="bold", color=text_color, pad=15)
plt.xlabel("Refund Rate (%)", color=text_color)
plt.ylabel("Route", color=text_color)
plt.grid(axis="x", linestyle="--", alpha=0.4, color=accent)
plt.gca().invert_yaxis()
plt.tight_layout()
plt.show()
```



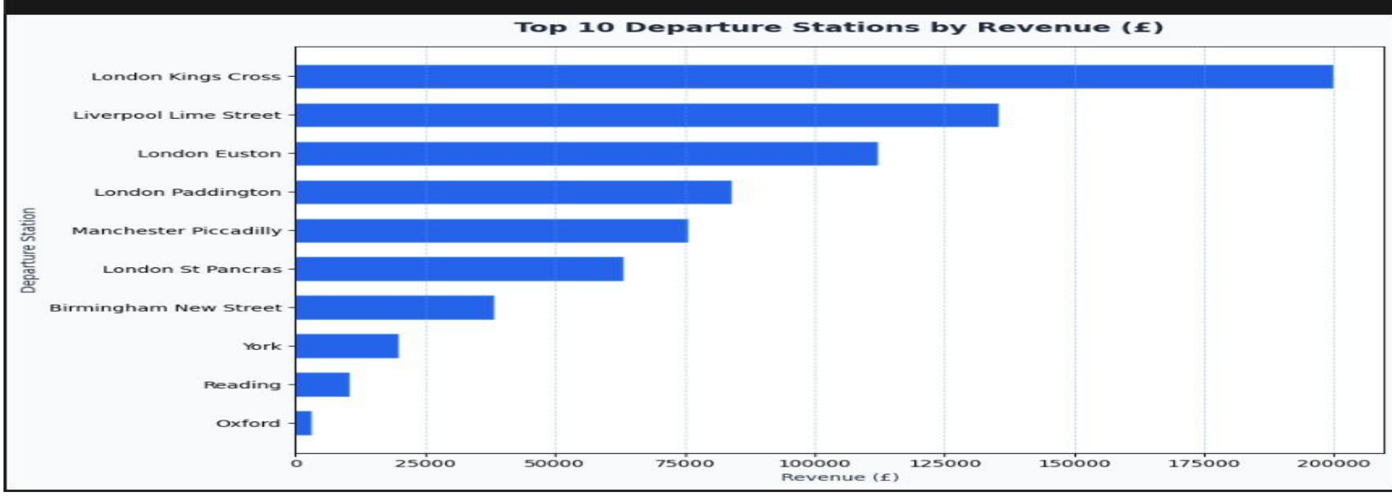
9- Which stations are the most crowded, based on ticket sales?

```python
passenger_count = df["Departure Station"].value_counts().head(10)
fig, ax = plt.subplots(figsize=(10,7), facecolor=background)
ax.set_facecolor(card_bg)
bars = ax.barh(passenger_count.index, passenger_count.values,
               color=primary, edgecolor=secondary, height=0.6)
ax.set_title("Most Crowded Departure Stations (Highest Ticket Sales)",
             fontsize=14, fontweight="bold", color=text_color, pad=15)
ax.set_xlabel("Number of Tickets Sold", fontsize=12, color=text_color)
ax.set_ylabel("Departure Station", fontsize=12, color=text_color)
ax.invert_yaxis()
ax.grid(axis="x", linestyle="--", alpha=0.4, color=accent)
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
ax.tick_params(axis="x", colors=text_color)
ax.tick_params(axis="y", colors=text_color)
plt.tight_layout()
plt.show()
```



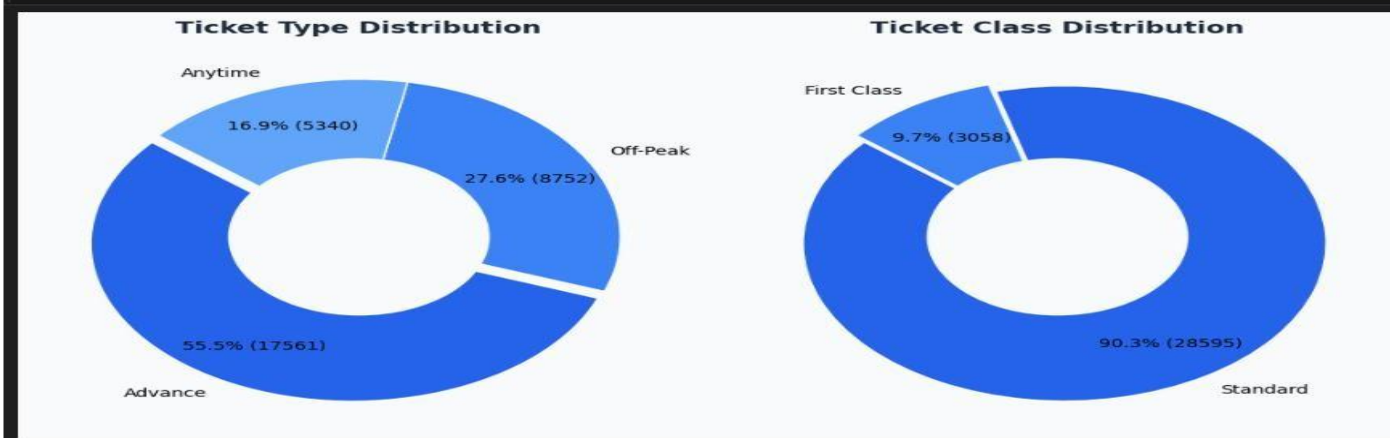
10- Which stations and routes generate the highest revenue?

```python
revenue_station = df.groupby("Departure Station")["Price"].sum().sort_values(ascending=False).head(10)
fig, ax = plt.subplots(figsize=(10,7), facecolor=background)
ax.set_facecolor(card_bg)
bars = ax.barh(revenue_station.index, revenue_station.values,
               color=primary, edgecolor=secondary, height=0.6)
ax.set_title("Top 10 Departure Stations by Revenue (£)",
             fontsize=14, fontweight="bold", color=text_color, pad=15)
ax.set_xlabel("Revenue (£)", color=text_color)
ax.set_ylabel("Departure Station", color=text_color)
ax.invert_yaxis()
ax.grid(axis="x", linestyle="--", alpha=0.4, color=accent)
plt.tight_layout()
plt.show()
```



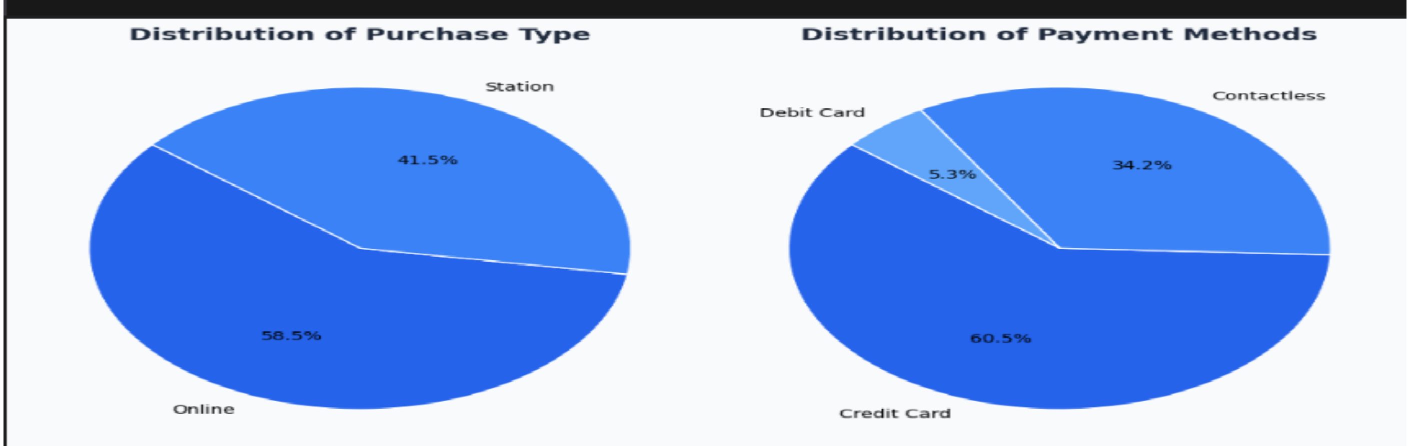
11- What are the most frequently purchased ticket types and classes overall?

```python
df["Ticket Type"] = df["Ticket Type"].fillna("Unknown")
df["Ticket Class"] = df["Ticket Class"].fillna("Unknown")

type_counts = df["Ticket Type"].value_counts()
class_counts = df["Ticket Class"].value_counts()

colors_type = [primary, secondary, accent, light_blue1, light_blue2][:len(type_counts)]
colors_class = [primary, secondary, accent, light_blue1, light_blue2][:len(class_counts)]

fig, axes = plt.subplots(1, 2, figsize=(10,7), facecolor=background)

axes[0].pie(type_counts, labels=type_counts.index, autopct=lambda p: f"{p:.1f}% ({int(p*sum(type_counts)/100)})",
            startangle=140, colors=colors_type, wedgeprops={'edgecolor': card_bg, 'linewidth':1},
            pctdistance=0.75, explode=[0.05 if i==0 else 0 for i in range(len(type_counts))])
centre_circle = plt.Circle((0,0),0.50,fc=background)
axes[0].add_artist(centre_circle)
axes[0].set_title("Ticket Type Distribution", fontsize=14, fontweight="bold", color=text_color)

axes[1].pie(class_counts, labels=class_counts.index, autopct=lambda p: f"{p:.1f}% ({int(p*sum(class_counts)/100)})",
            startangle=140, colors=colors_class, wedgeprops={'edgecolor': card_bg, 'linewidth':1},
            pctdistance=0.75, explode=[0.05 if i==0 else 0 for i in range(len(class_counts))])
centre_circle2 = plt.Circle((0,0),0.50,fc=background)
axes[1].add_artist(centre_circle2)
axes[1].set_title("Ticket Class Distribution", fontsize=14, fontweight="bold", color=text_color)
plt.tight_layout()
plt.show()
```



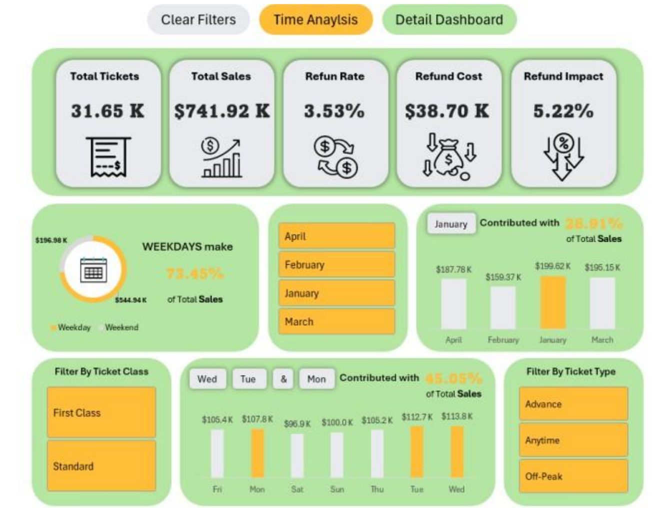
12- What is the distribution of payment methods (online vs. station purchase, card vs. cash)?

```python
df["Purchase Type"] = df["Purchase Type"].fillna("Unknown")
df["Payment Method"] = df["Payment Method"].fillna("Unknown")
df["Refund Request"] = df["Refund Request"].str.lower()
df["Journey Status"] = df["Journey Status"].str.lower()
purchase_counts = df["Purchase Type"].value_counts()
payment_counts = df["Payment Method"].value_counts()
colors_purchase = [primary, secondary][:len(purchase_counts)]
colors_payment = [primary, secondary, accent, light_blue1, light_blue2][:len(payment_counts)]

fig, axes = plt.subplots(1, 2, figsize=(10,7), facecolor=background)

axes[0].pie(purchase_counts, labels=purchase_counts.index, autopct="%1.1f%%",
            startangle=140, colors=colors_purchase, wedgeprops={'edgecolor': card_bg})
axes[0].set_title("Distribution of Purchase Type", fontsize=14, fontweight="bold", color=text_color)

axes[1].pie(payment_counts, labels=payment_counts.index, autopct="%1.1f%%",
            startangle=140, colors=colors_payment, wedgeprops={'edgecolor': card_bg})
axes[1].set_title("Distribution of Payment Methods", fontsize=14, fontweight="bold", color=text_color)
plt.tight_layout()
plt.show()
```

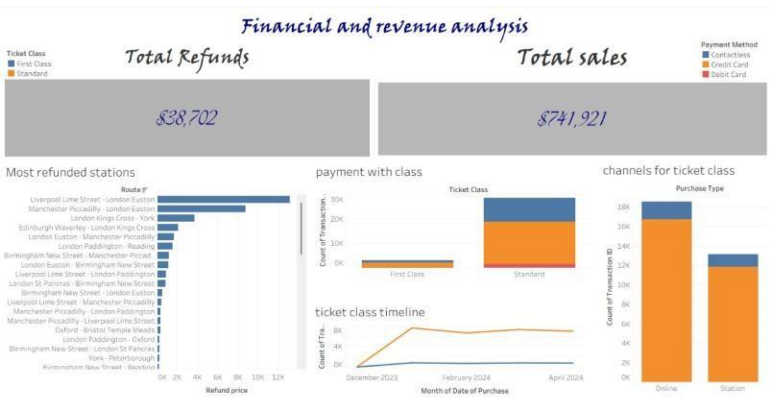


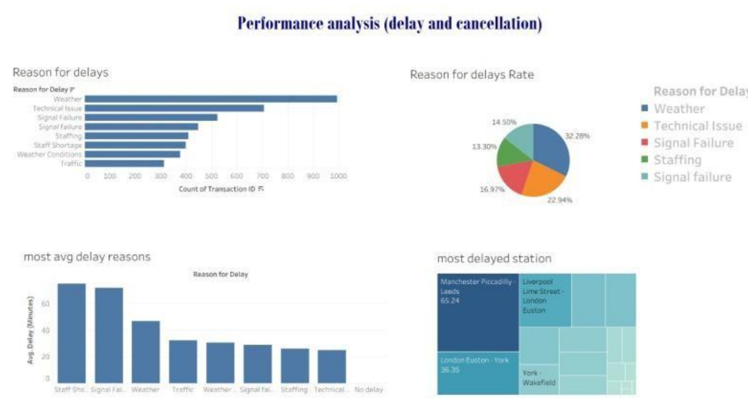
Dashboards

1- Excel Dashboards:



2- Tableau Dashboards:





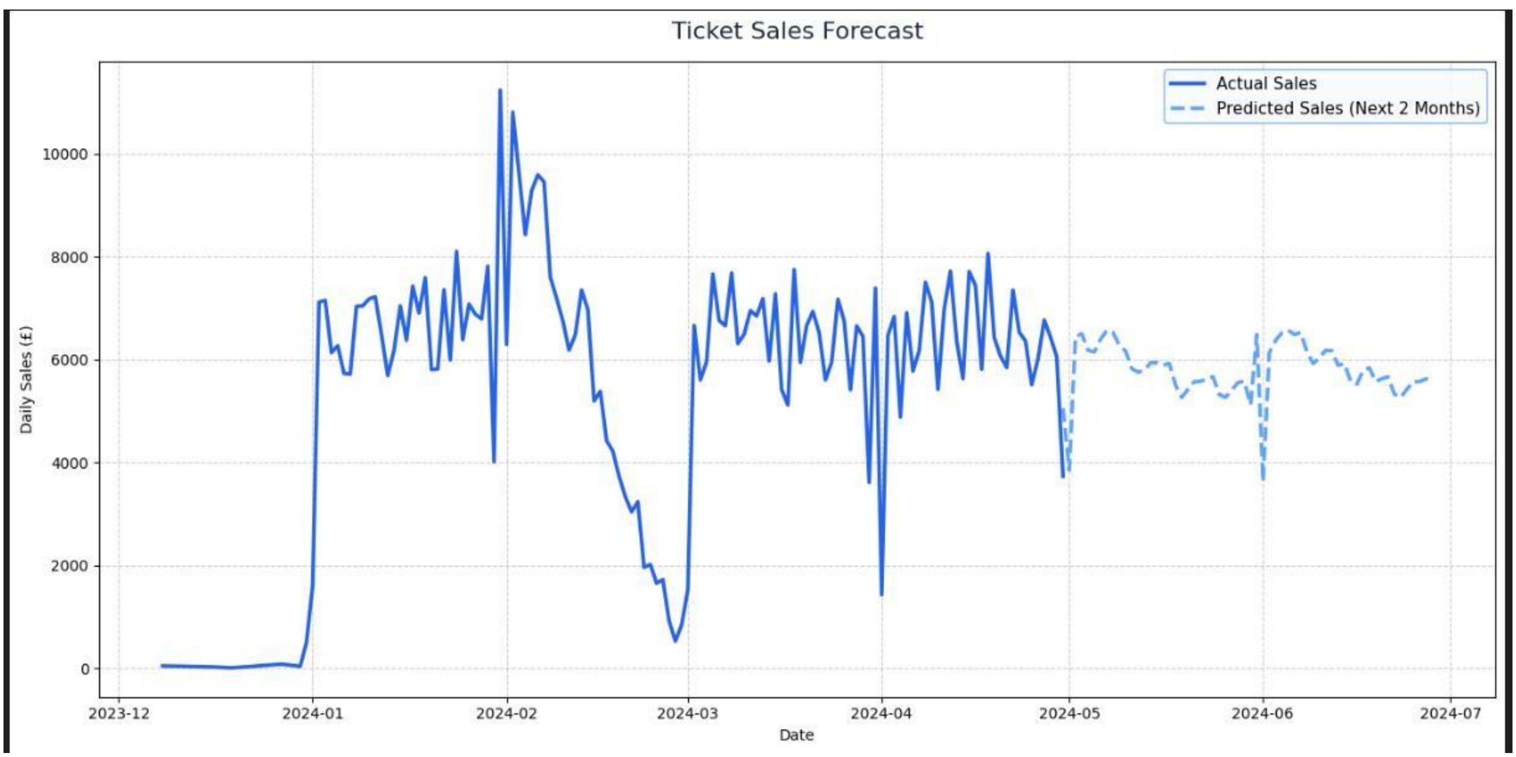
3- Python Dashboard:

Predictions

```python
train = daily_sales.iloc[:-10]
test = daily_sales.iloc[-10:]
features = ["day_num", "day_of_week", "month", "day_of_month", "rolling_7", "rolling_14", "lag_1", "lag_7"]
X_train, y_train = train[features], train["sales"]
X_test, y_test = test[features], test["sales"]

rf = RandomForestRegressor(n_estimators=300, random_state=42)
xgb = XGBRegressor(n_estimators=300, learning_rate=0.05, max_depth=5, random_state=42)

rf.fit(X_train, y_train)
xgb.fit(X_train, y_train)

ensemble_train = np.column_stack([rf.predict(X_train), xgb.predict(X_train)])
meta_model = LinearRegression()
meta_model.fit(ensemble_train, y_train)

ensemble_test = np.column_stack([rf.predict(X_test), xgb.predict(X_test)])
final_pred = meta_model.predict(ensemble_test)
```

```python
future_days = np.arange(len(daily_sales), len(daily_sales) + 60)
future_dates = pd.date_range(daily_sales["date"].max(), periods=60, freq="D")

future_df = pd.DataFrame({
    "date": future_dates,
    "day_num": future_days,
})
future_df["day_of_week"] = future_df["date"].dt.dayofweek
future_df["month"] = future_df["date"].dt.month
future_df["day_of_month"] = future_df["date"].dt.day
future_df["rolling_7"] = daily_sales["sales"].iloc[-7:].mean()
future_df["rolling_14"] = daily_sales["sales"].iloc[-14:].mean()
future_df["lag_1"] = daily_sales["sales"].iloc[-1]
future_df["lag_7"] = daily_sales["sales"].iloc[-7]

rf_future = rf.predict(future_df[features])
xgb_future = xgb.predict(future_df[features])
ensemble_future = np.column_stack([rf_future, xgb_future])
future_df["predicted_sales"] = meta_model.predict(ensemble_future)
```

```python
plt.figure(figsize=(14,7))
plt.plot(daily_sales["date"], daily_sales["sales"], label="Actual Sales", color=primary, linewidth=2.5)
plt.plot(future_df["date"], future_df["predicted_sales"], "--", label="Predicted Sales (Next 2 Months)", color=accent, linewidth=2.5)
plt.title("Ticket Sales Forecast", fontsize=16, color=text_color, pad=15)
plt.xlabel("Date")
plt.ylabel("Daily Sales (£)")
plt.legend(facecolor=background, edgecolor=accent, fontsize=11)
plt.grid(True, linestyle="--", alpha=0.6)
plt.tight_layout()
plt.show()
```



Stakeholder Analysis

	Key Interest(s)	Influence	Project Requirements
Executive Management (CEO, COO)	• **Main Target:** Reducing delays from 13.2% to 5%. • **Financials:** Minimizing revenue loss, overall profitability. • **Customer Satisfaction:** High-level brand perception.	High	• High-level dashboard summary. • Clear tracking of the 13.2% -> 5% target. • Executive summary of recommendations.
Operations Team (Station/Route Managers)	• **Root Causes:** *Why* are delays happening? • **Location:** *Which* stations and routes are the worst? • **Time:** *When* do delays occur?	High	• Detailed, filterable dashboard views for specific stations, routes, and times. • Insights into "Station/route imbalance."
Finance Department	• **Revenue:** How much revenue is lost from refunds? • **Analysis:** Revenue contribution by route/station..	Medium	• Accurate "Profit Loss" KPI. • Data on refund rates by cause. • Analysis of ticket sales patterns.
Maintenance & Technical Teams	• **Failures:** What are the most common "Technical Problems" causing delays?	Medium	• A prioritized list of technical failure-modes based on frequency and impact.
Marketing & Pricing Team	• **Demand:** "First class online vs. station," "demand on first class when crowded." • **Behavior:** Most sold ticket types, payment methods.	Medium	• Insights into customer purchasing behavior. • Data to support a dynamic pricing strategy (your point #5).
Data Analysis Team (Project Team)	• **Data Access:** Getting clean, reliable data. • **Scope:** A clear set of questions and KPIs. • **Tools:** Access to required software (SQL, Python, Tableau).	High	• Clear definitions of all KPIs and problems. • Access to all necessary data tables.

Acknowledgement

We would like to thank our instructor, Dr/ Yasser Abdelrahman, for their valuable guidance and continuous support throughout this six-month training journey. We also express our gratitude to the Ministry of Communications and Information Technology for providing this opportunity to gain practical experience, and to CLS Company for their supervision and contribution to our project's success. This experience has greatly enhanced our technical and professional skills.